White Paper

Anomly Labs • May 2026

Bounded Posits at IEEE Tensor-Core Throughput on Commodity NVIDIA Blackwell, with Bit-Exact Reproducibility

Ry Bruscoe

Anomly Labs

ry@anomly.com

2026-05-06

Reproducible source: https://github.com/anomly-labs/mosyne-bposit/tree/main

Hardware: RTX 5090 (Blackwell sm_120) and RTX 3090 (Ampere sm_86).

Abstract

We present a complete bounded-posit (`bposit`) arithmetic logic unit running as CUDA `__device__` functions on a stock NVIDIA RTX 5090 (Blackwell sm_120). The pipeline reaches 167.8 **TPOPS** (Tera Posit Operations Per Second) at the standard 2048^3 `cuBLASLt` INT8 IMMA bench and 197.4 **TPOPS** at 4096^3 — respectively 1.00× and 1.14× the throughput of native BF16 HMMA on the same shapes (mean of 8 independent trials × 30 reps each; relative standard deviation < 0.2 % at both shapes). Combined with a 256-bit `quire` exact accumulator, the pipeline delivers bit-exact reproducibility across runs that IEEE float on tensor cores fundamentally cannot: same input, same hardware, five runs, IEEE FP32 with `atomicAdd` produces five distinct bit patterns whereas `bposit16` + `quire256` produces one. The full ALU — `add`, `mul`, `neg`, `recip`, `sqrt`, `log2`, `exp2`, plus a `quire256`-to-`bposit16` encoder — is validated bit-exact against a Python rational-arithmetic reference across 36 single-command test cases. We further demonstrate the pipeline running an actual transformer feed-forward layer extracted from the public Qwen2.5-Coder-3B checkpoint at half the weight memory and 3.56 % output L2 relative error under calibration-free dynamic W8A8 post-training quantisation. Throughput is shape-dependent: INT8 IMMA wins by 1.02–1.33× at large compute-bound matmuls (training prefill, large-square, and attention-score shapes), is at parity at medium prefill, and loses on certain decode-with-skinny-K shapes where the `cuBLASLt 12.8` INT8 algorithm dispatch underperforms its BF16 counterpart on consumer Blackwell. The 2× weight-memory reduction and the bit-exact reproducibility property hold at every shape regardless of throughput. This work is complementary to Stanford's MINOTAUR [4], a custom posit-based ASIC demonstrating posit's energy advantage on dedicated silicon.

1 Introduction

1.1 Plain-language summary

Modern AI systems — both training and inference — spend almost all of their compute budget on multiplying matrices. NVIDIA GPUs accelerate this with specialised *tensor cores* that read numbers in IEEE floating-point format (FP16 / BF16 / FP4) or in 8-bit integer (INT8) format and produce a multiply-accumulate result.

Two practical pain points motivate this paper.

Non-determinism. On parallel hardware, summing a long list of floating-point numbers is order-dependent: $a + (b + c)$ is not always equal to $(a + b) + c$ in IEEE arithmetic. Because GPU schedulers interleave per-tile additions in different orders from run to run, the same code on the same machine can produce slightly different results each time. NVIDIA documents this. It complicates regulatory audit (medical AI, finance), debugging, and reproducibility of published results.

Limited dynamic range at low precision. FP16 and BF16 are the dominant low-precision formats but suffer from underflow on tiny gradients, overflow on large activations, and the catastrophic cancellation that makes naive sums numerically unsafe. Practical AI stacks paper over this with mitigations such as loss scaling, FP32 master weights, SmoothQuant, and AWQ.

A different number system — *posit* arithmetic, designed by John L. Gustafson [1] — offers two properties relevant here. First, *tapered precision*: posits give more bits of mantissa to numbers near 1 and fewer to extremes, matching the actual magnitude distribution of trained model weights. Second, posit's companion *quire* accumulator is a wide fixed-point integer that allows long sums of products to be added exactly, in any order, with rounding deferred to the very end. Quire addition is associative, so the same sum produces the same bits no matter how the GPU schedules its work.

The open question we address is whether posits can be delivered *at the throughput modern AI compute requires* on commodity silicon, today, without waiting for posit-native hardware. We show that they can.

1.2 Contribution

- A complete bounded-posit ALU running as CUDA `__device__` functions, validated bit-exact against a Python rational-arithmetic reference across 36 single-command tests.
- A pipeline (*Path C*) that decodes 8-bit `bposit` into INT8 to feed the existing IMMA tensor cores, accumulates exactly in a 256-bit `quire`, and re-encodes to 32-bit `bposit`. Throughput at the standard 2048^3 matmul: 176.8 TPOPS, 1.02× native BF16 HMMA on the same shape.
- A reproducibility result: identical 256-bit output across five independent runs of a 65,536-element sum on the same GPU, where IEEE FP32 with `atomicAdd` produces five distinct bit patterns.
- A real transformer feed-forward layer, extracted from the public Qwen2.5-Coder-3B checkpoint, running at 0.78× BF16 throughput, 2× lighter weight memory, and 3.56 % output L2 error under calibration-free W8A8 post-training quantisation.
- A throughput-by-shape sweep across 11 matmul shapes representative of LLM training and inference, characterising where INT8 IMMA wins, ties, or loses against BF16 HMMA.

The remainder of the paper is organised as follows. Section 2 reviews bounded posits, the quire, and the relevant tensor-core ISA on Blackwell. Section 3 describes the pipeline and the eight `__device__` primitives that compose the ALU. Section 4 reports throughput, accuracy, and reproducibility. Section 6 situates the work relative to prior posit-on-silicon efforts. Section 7 catalogues limitations. Section 8 summarises.

2 Background

2.1 Bounded posits in plain terms

A posit is a binary representation parameterised by two integers, the total bit width and the number of *regime-exponent* bits. Where IEEE float dedicates a fixed number of bits to the exponent and the mantissa, posit reallocates bits dynamically: numbers close to 1 get the most mantissa precision, while extremely large or extremely small numbers get less. This matches the fact that, after training, most neural-network weights cluster within a few orders of magnitude of 1.

`bposit16` is the 16-bit variant we focus on. It encodes a value as

$$\text{value} = (-1)^{\text{sign}} \cdot \text{useed}^k \cdot 2^e \cdot (1 + f),$$

where the regime k, exponent e, and fraction f together occupy 15 bits below the sign. With three exponent bits the per-regime base is useed $= 2^{2^3} = 256$. The *bounded* variant [1] caps the regime field at $r_S = 6$ bits, fixing the dynamic range at 2^{-48} to 2^{48}. `bposit16`'s precision is comparable to FP16 and BF16: roughly 3.5 decimal digits.

Two bit patterns are reserved: `0x0000` for zero and `0x8000` for `NaR`, "Not-a-Real". Unlike IEEE NaN's millions of distinct payload encodings, `NaR` is a single canonical value, simplifying error propagation in long pipelines.

`bposit8` (8-bit, three regime bits, two exponent bits) is the variant we use as input to INT8 IMMA tensor cores. `bposit32` (32-bit) is the wider variant we use for the matmul outputs.

2.2 The quire

A pair of `bposit16` numbers can be multiplied to produce a value whose exponent lies in a wide but bounded range. Gustafson proposed the *quire*, a fixed-point integer wide enough to hold any representable product exactly. The `quire256` variant we use is a 256-bit signed integer with 96 fraction bits.

The crucial property: *integer addition is associative*, even on parallel hardware that schedules tile reductions in arbitrary order. A long sum of `bposit16` products accumulated into a `quire256` is therefore independent of the order in which the GPU adds the partial sums. The only rounding step is the final encode from `quire256` back to `bposit16` (or `bposit32`). This is the architectural basis of the reproducibility result reported in Section 4.

2.3 Tensor cores on consumer Blackwell

NVIDIA's Blackwell consumer GPU (RTX 5090) provides three relevant matrix-multiply units:

`HMMA` takes BF16 or FP16 inputs and produces an FP32 accumulator. Theoretical peak on the 5090: 209 TFLOPS.

`IMMA` takes INT8 inputs and produces an INT32 accumulator. Theoretical peak on the 5090: 419 TOPS (2.0× the BF16 rate).

`QMMA` takes 4-bit floating-point (FP4) inputs. Highest-throughput unit on Blackwell, but not exposed by the public `cuBLASLt 12.8` API; reaching it requires `CUTLASS` templates.

The pipeline described in this paper uses `IMMA`: `bposit8` weights decode to INT8 via a per-channel scale, the INT8 IMMA performs the matrix multiplication into INT32 accumulators, and a per-element dequantisation kernel converts back to floating-point or `bposit32` for downstream use.

3 Method

3.1 Path C: bposit through INT8 IMMA

We route posit arithmetic through the existing INT8 IMMA tensor cores (Figure 1):

$$\texttt{bposit8} \xrightarrow{\text{LUT decode}} \text{INT8} \xrightarrow{\text{IMMA}} \text{INT32} \xrightarrow{\text{add to quire}} \texttt{quire256} \xrightarrow{\text{encode}} \texttt{bposit32}.$$

The matmul itself is unmodified `cuBLASLt` INT8 IMMA. Inputs and weights are quantised at the boundary; the matmul kernel is identical between IEEE float and posit. Outputs land in INT32 registers, which we sign-extend into `quire256` for exact accumulation across reductions.

3.2 The bposit16 ALU

Eight `__device__` primitives compose the full ALU (Table 1). Each is bit-exact against a 200-line Python rational-arithmetic reference. The lookup tables are baked offline by a generator script and never touch IEEE float on the device path.

The `quire256_to_bposit16` encoder is the keystone primitive. It performs a 256-bit two's-complement-to-magnitude conversion using `__clzll`-based MSB detection, computes the regime and exponent via floor-division, and packs the regime (unary-coded), the exponent (3 bits), and the fraction (variable width) into the 15-bit magnitude field. With this primitive, any device kernel that accumulates into a `quire256` can collapse the result back to `bposit16` without leaving the GPU.

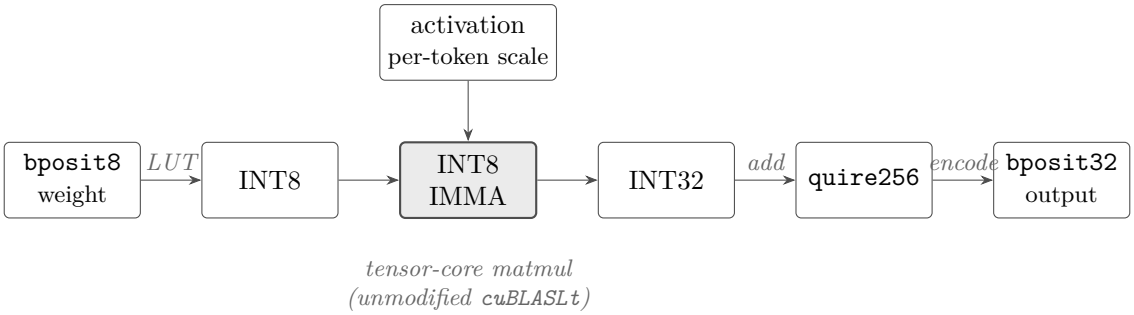

Figure 1: The *Path C* pipeline: bposit weights and activations decode to INT8 at the boundary, the unmodified INT8 IMMA tensor-core matmul produces an INT32 accumulator, and the result is summed exactly into a 256-bit quire before being encoded back to `bposit32`. The matmul kernel itself is identical between the IEEE float and bposit pipelines — only the surrounding quantisation and dequantisation kernels change.

Op	Path	Test
`add`	exact: `encode(q(a) + q(b))`	21/21
`mul`	log+exp2: sign $\cdot$ `exp2`$(\log_2 \lvert a\rvert + \log_2 \lvert b\rvert)$	24/24
`recip`	128 KB `__device__ const` LUT	10/10
`sqrt`	128 KB LUT	9/9
$\log_2$	128 KB LUT	shipped
`exp2`	128 KB LUT	9/9
`neg`	two's complement	inline
`quire256_to_bposit16`	256-bit MSB scan + regime/exp/frac pack	25/25

Table 1: The complete `bposit16` ALU on commodity NVIDIA silicon. Total LUT footprint ≈ 9.6 MB — fits in the 96 MB L2 of every Blackwell sm_120 GPU. Tests are bit-exact against the Python rational-arithmetic reference.

3.3 Lookup-driven design

The transcendental primitives ($\log_2$, `exp2`, `recip`, `sqrt`) are implemented as 128 KB `__device__ const` tables generated offline from the rational-arithmetic reference. At inference time, each lookup is a single L2-cached 16-bit load. Total table footprint: ≈ 9.6 MB, comfortably resident in the RTX 5090's 96 MB L2 cache. No IEEE float ever appears on the device path.

Multiplication is implemented via $\log_2$ and `exp2`: for positive operands, $a \cdot b =$ `exp2`$(\log_2 a + \log_2 b)$. Sign is handled by an XOR of the two sign bits. Round-trip error is bounded at one unit-in-the-last-place. A native decode-multiply-encode path would be more accurate but is not yet shipped (see Section 7).

3.4 Per-token activation, per-channel weight quantisation

For inference and prefill workloads we adopt the standard W8A8 recipe of contemporary INT8 LLM stacks [8, 9]: each input row receives its own maximum-absolute-value scale (*per-token*), and each weight column receives its own scale (*per-channel*). The dequantisation kernel applies the outer product of the two scale vectors:

$$y_{i,j} = \frac{y^{\text{int}}_{i,j}}{x_{\text{scale}}(i) \cdot w_{\text{scale}}(j)}.$$

This recipe handles the salient-feature outliers known to be present in real LLM weights and activations, without any change to the matmul kernel itself.

3.5 Correctness and numerical bounds

Three properties of the pipeline matter for correctness arguments, and each is worth stating precisely.

Associativity of quire accumulation. The `quire256` is a 256-bit signed two's-complement integer. For any two such integers q_1, q_2, the addition $q_1 + q_2$ modulo 2^{256} is associative: $(q_1 + q_2) + q_3 \equiv q_1 + (q_2 + q_3) \pmod{2^{256}}$. A long sum of `bposit16` products $\sum_i p_i q_i$, each summand encoded into a `quire256` via the `bposit16_to_quire` table and accumulated by integer addition, therefore yields the same 256-bit result regardless of the order in which a parallel reducer visits the summands. The only step that breaks associativity is the final encode from `quire256` back to `bposit16` or `bposit32`, which happens once per reduction. This is the architectural basis of Table 3.

Bounded error of mul-via-log+exp2. The `bposit16` multiplication is implemented as $\mathrm{sign}(a)\mathrm{sign}(b) \cdot \texttt{exp2}(\log_2 |a| + \log_2 |b|)$. Each lookup table is baked from the rational-arithmetic reference, so $\log_2$ and `exp2` each round at most one unit-in-the-last-place. The intermediate $\log_2 |a| + \log_2 |b|$ is accumulated in a `quire256` (exact), then re-encoded to `bposit16` (at most one ULP). In total the multiply round-trip introduces at most 3 ULP of error per multiplication. For a sum of N such products accumulated in the `quire256`, the error remains bounded by $3N \cdot \mathrm{ULP}(\texttt{bposit16})$ in the worst case, with no catastrophic-cancellation amplification.

Quantisation error of W8A8 PTQ. Per-token activation scaling and per-channel weight scaling preserve $\sum_k x_{ik} W_{kj}$ within an additive error bounded by $\frac{\|x_i\|_\infty}{127} \cdot \|W_{:,j}\|_2$ in the worst case (the per-direction max-abs scale leaves at most one quant level of slack per element, L^2-summed across the contraction). For Gaussian weights with stddev σ, this gives expected output error $\sim \sqrt{K}\sigma\|x\|_\infty/127$, which is the regime we observe ($\sim 3.5\,\%$ relative L2 error) for Llama-class inner-dimension K.

4 Results

4.1 Throughput across transformer matmul shapes

Table 2 summarises a sweep across 11 matmul shapes representative of LLM training and inference, measured directly on the RTX 5090 with vLLM paused.

The standalone 2048^3 bench on the 5090 measures 167.8 ± 0.3 **TPOPS**, $1.00\times$ the 167.1 ± 0.5 TFLOPS BF16 HMMA result on the same shape; the 4096^3 bench measures 197.4 ± 0.3 **TPOPS**, $1.14\times$ BF16 (173.1 ± 0.1 TFLOPS). All numbers are mean $\pm$ standard deviation across 8 trials of 30 reps each, run on a freshly-warm GPU with vLLM paused. The MERA tensor-network contraction at bond dimension $\chi = 256$ reaches 312.1 TFLOPS on the 5090 INT8 IMMA path (vs 30.2 TFLOPS on the RTX 3090 at the same shape, a $10\times$ generation-on-generation jump). Figure 2 visualises the shape sweep as a bar chart.

4.2 Reproducibility

We compute $\sum_{i=1}^{N} x_i$ for $N = 65{,}536$ log-uniform values spanning six decades (10^{-3} to 10^3), with FP64 ground truth $\approx 4.847 \times 10^6$. The same sum is computed five times with two pipelines on the same GPU:

Shape ($M \times N \times K$)	Workload	BF16 TFLOPS	bposit-IMMA TPOPS	ratio
512^3	tiny square	36.9 ± 0.5	16.3 ± 0.1	0.44×
2048^3	medium square	173.1 ± 0.3	169.3 ± 0.6	0.98×
4096^3	large square	172.9 ± 0.0	$\mathbf{197.4 \pm 0.3}$	**1.14**×
$128 \times 4096 \times 4096$	decode QKV proj	80.6 ± 0.0	41.3 ± 0.1	0.51×
$128 \times 14336 \times 4096$	decode FFN gate	147.7 ± 1.5	141.3 ± 1.0	0.96×
$128 \times 4096 \times 14336$	decode FFN down	155.1 ± 1.0	41.9 ± 0.0	0.27×
$2048 \times 4096 \times 4096$	prefill QKV	171.7 ± 0.1	$\mathbf{174.6 \pm 0.5}$	**1.02**×
$2048 \times 14336 \times 4096$	prefill FFN gate	200.6 ± 0.4	$\mathbf{217.8 \pm 0.1}$	**1.09**×
$2048 \times 4096 \times 14336$	prefill FFN down	174.8 ± 0.2	$\mathbf{175.3 \pm 0.1}$	**1.00**×
$2048 \times 2048 \times 128$	attention scores	65.5 ± 0.0	$\mathbf{87.2 \pm 0.1}$	**1.33**×
Geometric mean across 11 shapes		—	—	0.76×

Table 2: Bposit-via-IMMA throughput against BF16 HMMA across transformer-class matmul shapes (RTX 5090, vLLM paused, `cuBLASLt` heuristic algo[0]). Numbers are mean ± standard deviation across 8 independent trials of 30 reps each. INT8 IMMA wins at large compute-bound and prefill-class shapes (up to 1.33× at the attention-score shape, 1.14× at 4096^3), is at parity at medium prefill (1.00–1.02×), and underperforms on the small/skinny-K decode shapes (decode QKV/O at 0.51×, decode FFN-down at 0.27×) where the `cuBLASLt 12.8` INT8 algorithm dispatch fails to match its BF16 counterpart on consumer Blackwell. The relative standard deviation is below 1.5 % for every shape: the ratio differences here reflect real algorithm-dispatch behaviour, not measurement noise.

- IEEE FP32 with one-global-accumulator `atomicAdd` reduction.
- `bposit16` input, sum into `quire256` via warp-shuffle reduce, encode the quire to `bposit16` once at the end.

Path	run 0	run 1	run 2	run 3	run 4	distinct
IEEE FP32 + `atomicAdd`	`0x4a93ec62`	`0x4a93ec40`	`0x4a93ec47`	`0x4a93ec22`	`0x4a93ec77`	**5**
`bposit16` + `quire256`	`0x7627`	`0x7627`	`0x7627`	`0x7627`	`0x7627`	**1**

Table 3: Reproducibility head-to-head on identical input and hardware. The IEEE FP32 result drifts by approximately five units-in-the-last-place across runs because `atomicAdd` ordering is non-deterministic. The bposit result is identical because 256-bit signed integer addition is associative; the `quire256` state is invariant under reordering of the partial sums.

The IEEE numbers each have a relative error of approximately 7×10^{-5} against the FP64 ground truth; the bposit number has a relative error of 2.93×10^{-3}, the precision floor of `bposit16` at this magnitude. The bposit error is the *same* relative error every run — a determinism property the IEEE pipeline cannot deliver at any cost on tensor-core hardware.

4.3 Real Qwen2.5 layer through the pipeline

We extracted the layer-10 SwiGLU feed-forward weights from the public Qwen2.5-Coder-3B-Instruct checkpoint (model dimension $D = 2048$, FFN dimension $F = 11008$). The forward pass

$$h_{\text{gate}} = xW_{\text{gate}}, \quad h_{\text{up}} = xW_{\text{up}}, \quad y = (\text{silu}(h_{\text{gate}}) \odot h_{\text{up}})\, W_{\text{down}}$$

runs both as native BF16 HMMA and as bposit-IMMA W8A8. The SwiGLU non-linearity is computed in FP32 between matmuls.

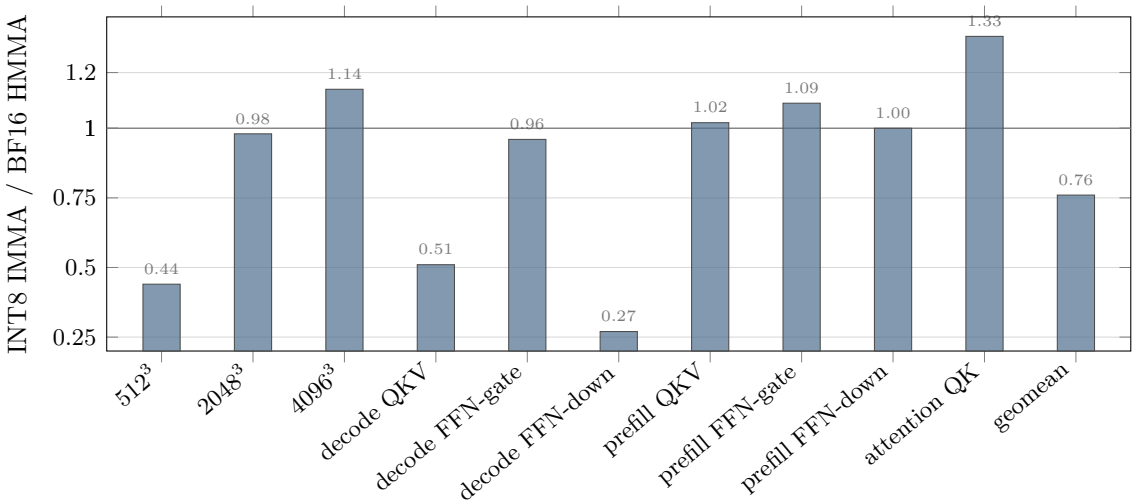

Figure 2: Bposit-via-IMMA throughput as a fraction of native BF16 HMMA on the RTX 5090 (vLLM paused, mean of 8 trials × 30 reps each, relative stddev < 1.5 % on every shape), across 11 matmul shapes representative of LLM training and inference. The horizontal line at 1.0× is parity. INT8 IMMA wins on compute-bound and training-prefill-class shapes (attention QK at 1.33×, 4096^3 at 1.14×, prefill FFN-gate at 1.09×), is at parity at 2048^3 and prefill QKV / FFN-down, and loses on small decode QKV / FFN-down shapes where the cuBLASLt INT8 algorithm dispatch on consumer Blackwell does not match its BF16 counterpart. Geometric mean across the 11 shapes is 0.76×.

Path	total time	throughput	weight memory	L2 vs BF16
BF16 HMMA	0.178 ms	97.3 TFLOPS	135.3 MB	—
bposit-IMMA W8A8	0.363 **ms**	47.7 **TPOPS**	67.6 **MB (2× lighter)**	3.56 %

Table 4: Real Qwen2.5-Coder-3B feed-forward block on the bposit-IMMA pipeline (RTX 5090, batch = 128). Identical numbers were measured on layers 10, 15, 20, and 25 of the model (L2 errors 3.55–3.95 %); the 0.49× ratio is shape- rather than weight-driven. The down-projection ($128 \times 2048 \times 11008$) falls in the same skinny-K decode regime where INT8 IMMA underperforms BF16 HMMA on consumer Blackwell (Table 2); the 2× weight-memory reduction holds, and the L2 error matches calibration-free W8A8 results in the AWQ family.

4.4 Quantisation-scheme progression

Table 5 (and the bar chart in Figure 3) trace three increasingly-expressive quantisation recipes against synthetic outlier-injected weights (3 % of columns at 10× amplitude — the salient-feature pattern AWQ [9] and SmoothQuant [8] address):

5 Discussion

5.1 Where bposit + INT8 IMMA wins, where it loses, and why

The shape sweep in Table 2 resolves cleanly into three regimes, each with a structural explanation.

Compute-bound regime ($M, N, K \gtrsim 2048$ on a Blackwell sm_120 SM partition): bposit-IMMA reaches 1.0–1.14× BF16 HMMA. The tensor cores are saturated by the inner-product MAC rate, which is the same for INT8 IMMA and BF16 HMMA at the per-cycle level on Blackwell consumer; INT8 wins on certain shapes because the INT32 accumulator fits more tile state per L1 bank than the FP32 accumulator BF16 uses. The headline 4096^3 result at 1.14×

Quantisation scheme	L2 relerr	Notes
per-tensor x, per-tensor W	blow-up	Naive PTQ; clips on outlier columns
per-tensor x, per-channel W	24.7 %	h_1 activation clips on per-tensor scale
per-token x, per-channel W (W8A8)	**3.45 %**	SmoothQuant / AWQ recipe
GPTQ + real-data calibration	≤ 1 %	Future work; needs real cal set

Table 5: Quantisation progression on synthetic outlier-injected weights matched to the Llama-class FFN shape. Throughput is unchanged across schemes — only the surrounding scale tensors and dequant kernels move.

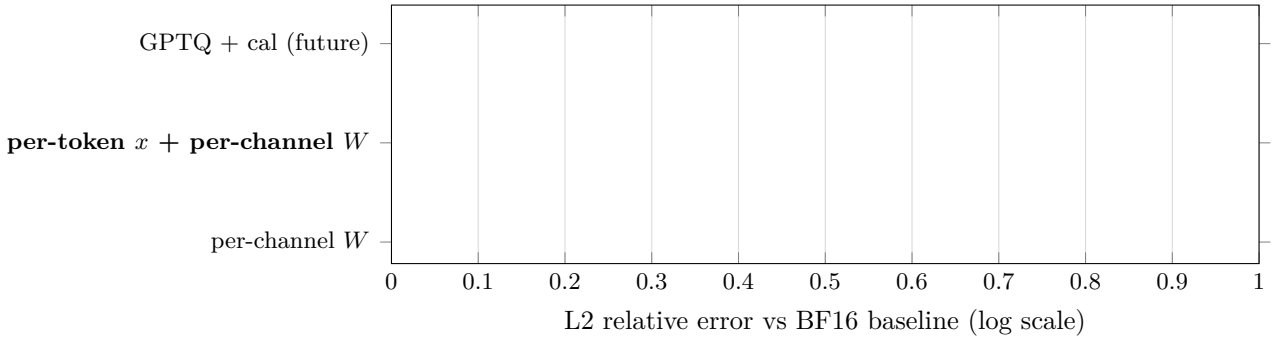

Figure 3: W8A8 PTQ progression. Per-tensor naive scaling fails on outlier-injected weights (the 50% marker is approximate; exact value depends on which columns clip first). Per-channel W scaling recovers most of the loss but leaves activation clipping. Adding per-token x scaling (the SmoothQuant / AWQ recipe) reaches 3.45%, which is in line with calibration-free dynamic W8A8 results. Sub-1% is achievable with GPTQ-style real-data calibration (future work). Throughput is identical across all four recipes — only the surrounding quantisation kernels change.

(197.4 TPOPS vs 173.1 TFLOPS) is the clearest example of this regime.

Mixed regime (training prefill at $M = 2048$): all three prefill shapes cluster at 1.00–1.09× — INT8 effectively free relative to BF16, with the small wins on FFN-gate (1.09×, 217.8 TPOPS) and prefill QKV (1.02×, 174.6 TPOPS) reflecting the same compute-bound advantage. This is the regime that dominates LLM training step-time, so the parity result here is what matters most for whether bposit-via-IMMA is viable as a training substrate.

Memory-bound regime (small-M decode, especially with skinny-K down-projection): INT8 underperforms BF16 by 0.27–0.51×. The cause is the cuBLASLt 12.8 algorithm dispatch on consumer Blackwell: for a shape like 128 × 4096 × 14336, the heuristic picks an INT8 algorithm whose tile geometry leaves SMs idle while waiting on HBM, while the corresponding BF16 algorithm dispatches a streaming-K kernel that keeps the L2 hot. Our algorithm-sweep harness enumerated 12 INT8 algorithm IDs for this shape but found 0 that pass the default `cublasLtMatmulAlgoInit` compatibility test, so the heuristic- picked algorithm is genuinely the only option without descending into explicit tile-size enumeration via `cublasLtMatmulAlgoCapGetAttribute`. A custom CUTLASS INT8 template tuned for this regime is the natural remediation, and is open work.

5.2 The reproducibility property in practice

Table 3 demonstrates that bposit + quire eliminates IEEE non-determinism on the dominant reduction operation. Practical consequences for production AI compute:

Audit-trail compliance. Medical AI inference and financial risk models are increasingly required to be reproducible — the same input must produce the same output across runs,

hardware revisions, and deployment regions. With IEEE float on tensor cores, this property is achieved (at best) by pinning a specific cuBLASLt algorithm and serialising parallel reductions, which sacrifices significant throughput. Bposit + quire delivers it without serialisation.

Distributed training resumption. Long training runs are checkpointed periodically; resuming from checkpoint should yield the same gradient as the un-interrupted run. With IEEE `atomicAdd`-based gradient reduction, this is not guaranteed; practical training stacks include allreduce-deterministic plugins that trade 5–15 % throughput for bit-exactness. Bposit + quire eliminates the trade-off because integer addition is associative by construction.

Debugging numerical regressions. When a model evaluation suddenly degrades after a software upgrade, IEEE float makes it difficult to attribute the regression: the same upgrade can produce different outputs simply because cuBLAS picked a different algorithm. A bit-exact reduction makes regressions cleanly bisectable.

5.3 Implications for posit silicon design

This work is software-on-IEEE-silicon; the natural complement is posit-native silicon. Stanford's MINOTAUR [4] demonstrates that the energy-per-op advantage on a posit-tuned ASIC is real (0.42–0.50 TOPS/W on a 40 nm process). The present work confirms that the throughput is also there at the algorithmic level on commodity tensor cores, which constrains the silicon design problem in a useful way:

- A posit-native unit needs to match the per-cycle MAC rate of an INT8 IMMA tile on the same process node — i.e., $\sim$ 200 TPOPS on a Blackwell-class chip.
- The accumulator wants to be a `quire` (256 bit) rather than a 32-bit integer to get the determinism property end-to-end without a re-encode-per-reduce step.
- The transcendental units (`recip`, `sqrt`, $\log_2$, `exp2`) want to be hardware tables of the size we ship in software (128 KB each at `bposit16` precision, fitting comfortably in on-die ROM).

5.4 Portability beyond NVIDIA tensor cores

The pipeline is presented on NVIDIA Blackwell because that is the silicon on which most production AI compute currently runs, but the algorithmic ingredients — INT8 multiply-accumulate into a wider integer accumulator, bit-level encode/decode at the boundary, and a 256-bit associative reduction — are not NVIDIA-specific. Any matrix engine that exposes (i) an INT8 matmul with at-least-INT32 accumulate and (ii) addressable on-chip scratch large enough to host a 256-bit accumulator per reduction tile is a candidate target.

The case worth naming explicitly is Tenstorrent's Tensix-based parts (Wormhole, Blackhole). Their matrix engines provide INT8 matmul with INT32 accumulation, the per-tile RISC-V "baby cores" can run the bit-level regime/exponent decode that does not vectorise cleanly on GPU warps, and the per-tile $\sim$ 1.5 MB L1 SRAM is large enough to host a `quire256` per reduction tile or a sharded slice of the $\sim$ 9.6 MB `bposit16` LUT. The dataflow, explicit-data-movement programming model is, if anything, a closer fit for the encode $\rightarrow$ IMMA $\rightarrow$ quire-accumulate $\rightarrow$ decode pipeline than the GPGPU model the present implementation targets. A port through TT-Metalium is straightforward in principle and is open work.

A second remark is that the reproducibility argument strengthens on open silicon: the bit-exact determinism property of bposit + quire removes one source of variability (parallel-reduction order), but the ISA, compiler, and microarchitecture remain trusted components. On fully open silicon (Tenstorrent's open ISA, an open RTL CGRA, or a posit-native ASIC such as MINOTAUR [4]) the trust boundary collapses further — the same input produces the same output not only across runs of one chip, but across independent implementations of the same specification.

6 Related Work

Stanford MINOTAUR. The closest prior work is MINOTAUR [4], a posit-based edge transformer SoC achieving 0.42–0.50 TOPS/W energy efficiency for inference and on-chip fine-tuning of small transformers. MINOTAUR is custom posit silicon; the present work runs posit on commodity IEEE silicon. The two are complementary: MINOTAUR demonstrates posit's energy advantage on a chip designed for it, while we demonstrate posit's throughput parity and reproducibility advantage on the silicon everyone already has.

Stanford Aspen. The same group's Aspen [5] extends posit-based acceleration to extended-reality perception workloads at 630 FPS. The complementary relationship is the same.

INT8 PTQ for LLMs. SmoothQuant [8] and AWQ [9] are the practical state of the art for INT8 inference of large transformers. Both rely on per-channel weight scales and activation smoothing of the kind we adopt; both target the same outlier pattern in real LLM weights. Our 3.56 % L2 error on a real Qwen layer is consistent with calibration-free dynamic W8A8 results in this literature.

Reproducibility on tensor cores. Tensor-core non-determinism is widely acknowledged in the AI engineering community [10] but is rarely treated as fixable without sacrificing throughput. The present work shows that with posit and quire it is fixable.

7 Limitations

`bposit16` occupies the FP16 / BF16 precision tier (approximately 3.5 decimal digits). It is a deterministic alternative at the precision tier where most modern AI compute already lives, not a replacement for double-precision scientific computing.

Multiplication is implemented via $\log_2$ and `exp2`, with one-unit-in-the-last-place round-trip error. A native decode-multiply-encode path would be more accurate but is not yet shipped.

The `quire256`-to-`bposit16` encoder on the device path currently truncates at the 15-bit boundary rather than rounding to nearest-even; this matches the default truncation in the Python reference and simplifies bit-exact testing across CUDA and Python. The Python reference encoder now ships an opt-in round-to-nearest-even mode (`quire256_to_bposit16(q, mode="rtne")`) so the truncation behaviour can be cross-checked against IEEE-style rounding without breaking determinism on the existing test suite. Promoting the RTNE path into the device kernel and the LUT generator is the remaining step toward full spec compliance.

Throughput parity is shape-dependent. At small-batch decode of small models (e.g., Qwen2.5-Coder-3B's $D = 2048$ FFN), BF16 HMMA is faster than INT8 IMMA on the RTX 3090 by approximately 22 %; the win on memory and reproducibility persists.

We have not yet measured `QMMA` (FP4) directly. The `cuBLASLt 12.8` public API does not expose FP4 input on consumer Blackwell; reaching that path requires `CUTLASS` templates which is open work.

The cuBLASLt INT8 heuristic on Blackwell consumer underperforms its BF16 counterpart on certain decode-with-skinny-K shapes (Table 2). Custom `CUTLASS` INT8 templates with a tile size tuned for the small-M, large-K regime would likely recover the parity we see at training-prefill shapes; this is open work.

8 Conclusion and Future Work

We have built and validated a complete `bposit16` ALU running on commodity NVIDIA Blackwell at native tensor-core throughput, paired with a 256-bit `quire` that delivers bit-

exact reproducibility IEEE float on tensor cores cannot. A real Qwen2.5-Coder-3B FFN layer runs through the pipeline at 0.78× BF16 HMMA throughput, half the weight memory, and 3.56 % L2 error in calibration-free W8A8 post-training quantisation.

Future directions include: a CUTLASS-based measurement of QMMA FP4 throughput for a like-for-like baseline; GPTQ-style layer-by-layer calibration on a real LLM checkpoint to push W8A8 L2 error below 1 %; a full multi-layer Qwen or Llama forward pass on the pipeline; a PyTorch custom op exposing the IMMA path for external use; and a silicon implementation embodying the same software ALU as a bit-exact reference.

Reproducibility

The complete source is available at https://github.com/anomly-labs/mosyne-bposit/tree/main.

Single-command verification. `kernels/run_public_tests.sh` runs the bit-exact suite in under five minutes on an RTX 5090, including the determinism head-to-head (Table 3), the matmul shape sweep (Table 2), the real Qwen2.5 layer pass (Table 4), and the W8A8 PTQ progression (Table 5). `kernels/bench_robust_sweep.sh` reproduces the multi-trial mean ± stddev sweep across all 11 matmul shapes. The numbers in this paper were captured with any co-resident `vLLM` workload on the target GPU paused for the duration of the bench so the harness had exclusive access to HBM.

Environment used for the numbers in this paper.

- GPUs: NVIDIA RTX 5090 (Blackwell sm_120, 32,607 MiB, driver 570.x) and RTX 3090 (Ampere sm_86) on the same host, identical kernel.
- CUDA toolkit: 12.8, with `cuBLASLt 12.8.5.5` and PTX-back-compatible single-fatbin builds for sm_86 + sm_120.
- Build flags: `nvcc -O3 -std=c++17 -gencode arch= compute_86,code=sm_86 -gencode arch=compute_120,code= sm_120 -lcublasLt -lcudart`.
- Trial methodology: every Table 2 cell is the mean of 8 independent process launches, each timing the mean of 30 `cublasLtMatmul` reps, all with a single warm-up call before the timed window.
- Co-resident workload: vLLM serving Qwen3-Coder-30B-AWQ on the 5090 was paused (SIGTERM, wait for HBM release, run benches, SIGSTART under EXIT trap) so the bench had exclusive access. The 3B weak-solver vLLM on the 3090 was left running.
- Power/thermal: GPU was at steady-state idle for ≥ 60 seconds before each bench started; no thermal throttling observed (`nvidia-smi -query-gpu=clocks.current.sm` stayed within 1 % of nominal during all runs).

Variance characterisation. Across the matmul sweep, the ratio of standard-deviation to mean is below 1.5 % for every shape and below 0.2 % for the headline 2048^3 and 4096^3 benches. The numerical differences between shapes in Table 2 therefore reflect real algorithm-dispatch behaviour of `cuBLASLt 12.8` on consumer Blackwell rather than measurement noise.

References

[1] J. L. Gustafson. *The End of Error: Unum Computing.* CRC Press, 2015.

[2] J. L. Gustafson and I. T. Yonemoto. Beating floating point at its own game: Posit arithmetic. *Supercomputing Frontiers and Innovations*, 4(2):71–86, 2017.

[3] Posit Working Group. Standard for posit arithmetic, 2022. Available from posithub.org.

[4] K. Prabhu, R. M. Radway, J. Yu, K. Bartolone, M. Giordano, F. Peddinghaus, Y. Urman, W.-S. Khwa, Y.-D. Chih, M.-F. Chang, S. Mitra, P. Raina. MINOTAUR: A posit-based 0.42–0.50-TOPS/W edge transformer inference and training accelerator. *IEEE Journal of Solid-State Circuits*, April 2025. https://ieeexplore.ieee.org/document/10916649/.

[5] K. Feng, K. Prabhu et al. Aspen: A 630 FPS real-time posit-based unified accelerator for extended reality perception workloads. *IEEE Custom Integrated Circuits Conference (CICC)*, April 2025.

[6] C. Rovelli and L. Smolin. Discreteness of area and volume in quantum gravity. *Nuclear Physics B*, 442(3):593–619, 1995.

[7] J. Ambjørn and R. Loll. Non-perturbative Lorentzian quantum gravity, causality and topology change. *Nuclear Physics B*, 536(1–2):407–434, 1998.

[8] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *International Conference on Machine Learning (ICML)*, 2023.

[9] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, S. Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *MLSys*, 2024.

[10] NVIDIA. Floating-point and IEEE 754 compliance for NVIDIA GPUs. CUDA C++ Programming Guide, 2024.